A

Project Report on

Delivery Pipeline and Identifying Bottlenecks

Submitted for partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

K. Ayush - 24K81A05M7

Under the Guidance of

Mr. M. Dileep Kumar

Assistant professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ST. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500

100 www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOUMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified

Dhulapally, Secunderabad-500 100

www.smec.ac.in

CERTIFICATE

This is to certify that the project titled **"Delivery Pipeline and Identifying Bottlenecks"** is being submitted by **K. Ayush (24K81A05M7)** in fulfillment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of Guide	Signature of HOD
Mr. M. Dileep Kumar	**Mr. D. Krishna Kishore**
Assistant Professor	Associate professor & HOD
Department of CSE	Department of CSE

Internal Examiner **External Examiner**

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOUMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified

Dhulapally, Secunderabad-500 100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of **"Bachelor of Technology in Department of Computer Science & Engineering"**, session: 2024 - 2028**, St. Martin's Engineering College**, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work entitled **"Delivery Pipeline and Identifying Bottlenecks"** is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

K. Ayush (24K81A05M7)

Acknowledgement

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. Ravindra**, Director & Principal, St. Martin's Engineering collage Dhulapally, for permitting us to undertake this project.

I am also thankful to **Mr**. **D. Krishna Kishore** Head of the Department, Computer science and engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project.

I would like to express my sincere gratitude and indebtedness to my project supervisor **Mr. M. Dileep Kumar** Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, I express thanks to all those who have helped us successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

K. Ayush(24K81A05M7)

CONTENTS

1. ABSTRACT ----------01

2. INTRODUCTION ----------02

3. LITERATURE SURVEY ----------03

4. SYSTEM REQUIREMENTS ----------04

5. FLOW CHART ----------05

6. ALGORITHM ----------06

7. SOURCE CODE ----------07

8. EXPERIMENTAL RESULTS ----------11

9. CONCLUSION ----------12

10. FUTURE ENHANCEMENT ----------13

11. REFERENCES ----------14

1. ABSTRACT

DevOps is a software development approach that combines development and operations practices to improve collaboration, automation, and software delivery. A delivery pipeline is an important component of DevOps that automates the process of moving software changes from development to deployment.The project titled **"Delivery Pipeline and Identifying Bottlenecks"** focuses on analyzing pipeline execution time and identifying stages that cause delays in the software delivery process.The proposed system uses Python to simulate different delivery pipeline stages, including Build, Test, Package, and Deploy. The execution time of each stage is measured using Python's time module. The measured values are stored and compared to identify the stage with the highest execution time.The system demonstrates how bottleneck identification can help developers understand pipeline performance and determine which stages require further investigation and optimization. For example, if the testing stage takes longer than the other stages, it can be identified as a potential bottleneck.The project introduces important DevOps concepts such as continuous integration, continuous delivery, automation, execution-time measurement, and performance monitoring. It also explains how the proposed approach can be integrated with tools such as Git and Jenkins to analyze real software delivery pipelines.The main objective is to reduce delivery delays, improve pipeline efficiency, minimize manual effort, and support reliable software development.The project provides a basic foundation for developing advanced monitoring systems that can analyze historical pipeline performance, generate alerts, and recommend optimization strategies.

UGC AUTONOMOUS

2. INTRODUCTION

Software development involves several activities, including writing source code, building applications, testing functionality, and deploying software. Performing these activities manually can consume considerable time and may introduce human errors. DevOps addresses these challenges by combining development and operations practices and automating repetitive software delivery tasks. Continuous Integration and Continuous Delivery are important DevOps practices that help teams deliver software changes efficiently. A delivery pipeline is a sequence of automated stages through which software passes before being released. A typical pipeline includes source code management, building, testing, packaging, and deployment. However, even an automated pipeline can experience delays. Slow compilation, lengthy test execution, dependency downloads, limited computing resources, and inefficient deployment scripts can increase the total delivery time.These delays are known as bottlenecks. Identifying bottlenecks helps development teams understand which stages require improvement.

The proposed system uses a Python program to simulate a delivery pipeline. Each stage is assigned a different execution delay, and the program measures the time taken by each stage. After execution, the program displays the duration of every stage and identifies the stage with the maximum measured time. The project provides a basic implementation of pipeline performance analysis. It can be extended to real CI/CD environments using Jenkins, GitHub, automated testing tools, and monitoring dashboards. This system is useful for software development teams, DevOps engineers, testing teams, and organizations that need to improve software delivery efficiency.

UGC AUTONOMOUS

3. LITERATURE SURVEY

The increasing demand for faster and more reliable software releases has encouraged organizations to adopt DevOps practices and automated delivery pipelines. Traditional software delivery methods often depend on manual building, testing, and deployment. These activities can result in delays, inconsistent processes, and difficulties in identifying the causes of performance problems. Continuous Integration is a software development practice in which code changes are integrated into a shared repository frequently. Automated builds and tests help identify errors early in the development process. Continuous Delivery extends this approach by preparing software changes for release through an automated pipeline. It reduces manual intervention and improves the consistency of software releases. Jenkins is an automation server widely used for implementing CI/CD pipelines. It supports pipeline stages, build execution, automated testing, and integration with version control systems. Its execution logs and stage information can help developers investigate performance issues. Git is a distributed version control system used to manage source code and track changes. It supports collaboration among developers and provides the foundation for triggering automated pipeline activities. Python provides useful functions for measuring execution time and analyzing data. Its standard library includes timing functions that can be used to calculate the duration of individual operations. Existing CI/CD tools provide automation and execution information, but identifying the actual cause of a delay may require additional analysis. The proposed project uses stage-wise time measurement to identify the stage with the highest execution duration. This provides a simple starting point for understanding pipeline performance. For example, if testing takes significantly longer than building and packaging, the testing process can be investigated for inefficient test cases or resource limitations. In conclusion, DevOps automation tools and performance measurement techniques provide a foundation for improving delivery pipeline efficiency. The proposed project demonstrates these concepts through a simple Python-based implementation.

4. SYSTEM REQUIRMENTS

4.1 Hardware Requirements

- **Processor:** Intel Core i3 or equivalent.
- **RAM:** Minimum 4 GB; 8 GB recommended.
- **Storage:** At least 5 GB of available disk space.
- **Display:** Standard laptop or desktop monitor.
- **Network:** Internet connection for downloading development tools and packages.

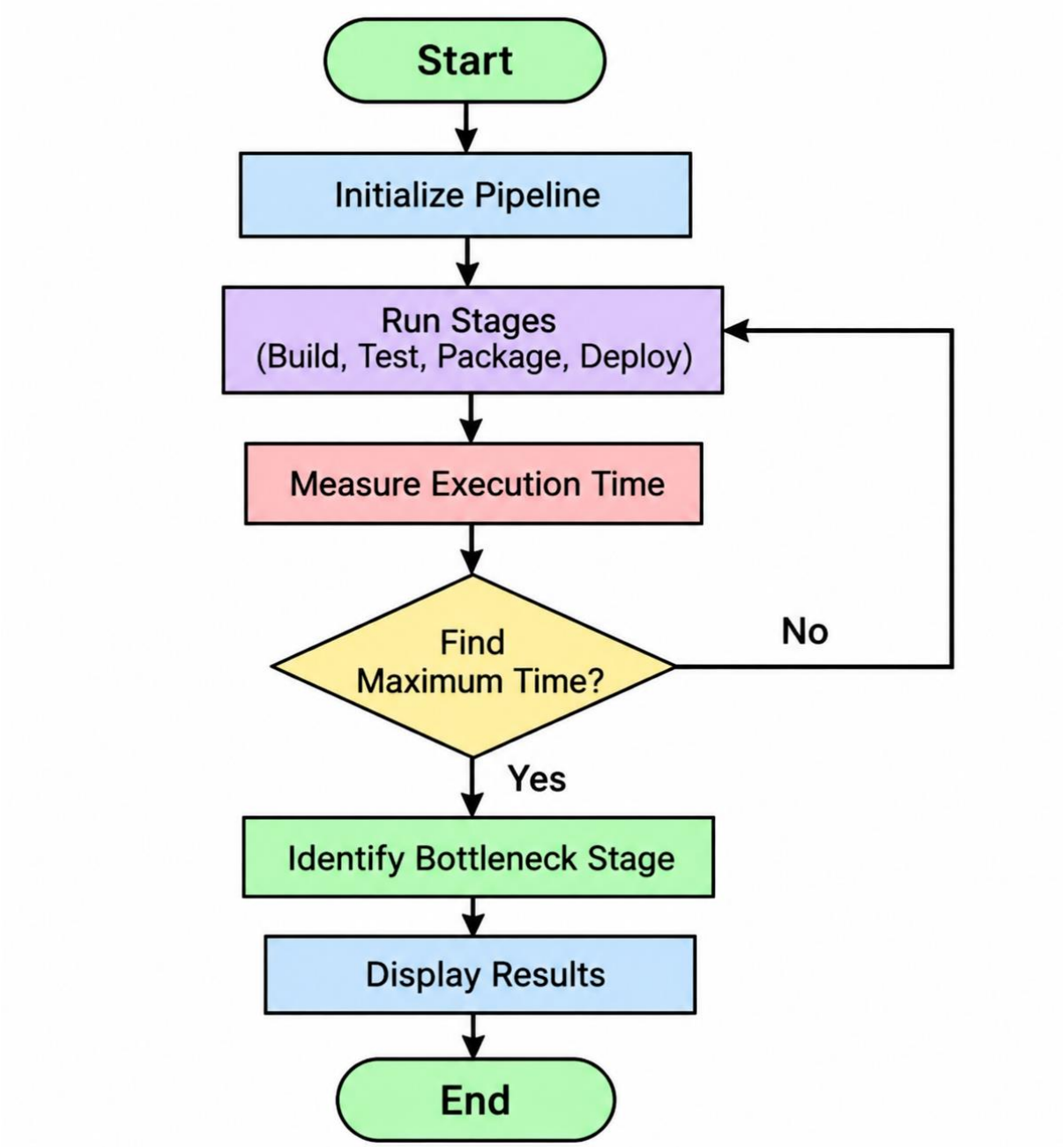
4.2 Software Requirements

- **Operating System:** Windows 10/11 or Linux.
- **Programming Language:** Python 3.
- **Code Editor:** Visual Studio Code or any text editor.
- **Version Control:** Git.
- **Automation Tool:** Jenkins for future or extended CI/CD integration.
- **Terminal:** Command Prompt, PowerShell, or Linux Terminal.
- **Browser:** Any modern web browser for accessing Jenkins.

4.3 Functional Requirements

1. Define the stages of a delivery pipeline.
2. Execute each stage in sequence.
3. Measure the execution time of each stage.
4. Store the measured execution times.
5. Compare the durations of different stages.
6. Identify the stage with the maximum execution time.

5. FLOW CHART


Start
Initialize Pipeline
Run Stages
(Build, Test, Package, Deploy)
Measure Execution Time
Find
Maximum Time?
No
Yes
Identify Bottleneck Stage
Display Results
End

6. ALGORITHM

1. Start the program.

2. Define the delivery pipeline stages: Build, Test, Package, and Deploy.

3. Assign a simulated execution delay to each stage.

4. Initialize an empty dictionary to store execution times.

5. Select the first pipeline stage.

6. Record the starting time using the Python timing function.

7. Execute the selected stage.

8. Record the ending time.

9. Calculate the execution time by subtracting the starting time from the ending time.

10. Store the execution time with the corresponding stage name.

11. Repeat Steps 5–10 until all pipeline stages are completed.

12. Compare the recorded execution times.

13. Identify the stage with the maximum execution time.

14. Display the execution time of every stage.

15. Display the stage identified as the primary potential bottleneck.

16. Stop the program.

UGC AUTONOMOUS

7. SOURCE CODE

```
import time
from datetime import datetime
stage_times = {}
stage_status = {}
def display_header():
    print("=" * 60)
    print("DELIVERY PIPELINE AND IDENTIFYING BOTTLENECKS")
    print("=" * 60)
    print("Start Time:", datetime.now())
    print()
def run_stage(name, delay):
    print("\nStarting", name, "Stage...")
    start = time.perf_counter()
    time.sleep(delay)
    end = time.perf_counter()
    stage_times[name] = end - start
    stage_status[name] = "SUCCESS"
    print(name, "completed successfully!")
    print("Execution Time:", round(stage_times[name], 2), "seconds")
def build_stage():
    print("\nPreparing application files...")
    run_stage("Build", 2)
def test_stage():
    print("\nExecuting application tests...")
```

```
    run_stage("Test", 4)
def package_stage():
    print("\nCreating application package...")
    run_stage("Package", 1)
def deploy_stage():
    print("\nDeploying application...")
    run_stage("Deploy", 3)
def execute_pipeline():
    print("\nPipeline execution started.")
    build_stage()
    if stage_status["Build"] == "SUCCESS":
        test_stage()
    else:
        print("Build failed!")
        return
    if stage_status["Test"] == "SUCCESS":
        package_stage()
    else:
        print("Testing failed!")
        return
    if stage_status["Package"] == "SUCCESS":
        deploy_stage()
    else:
        print("Packaging failed!")
        return
```

```
def calculate_statistics():
    total = sum(stage_times.values())
    average = total / len(stage_times)
    fastest = min(stage_times, key=stage_times.get)
    slowest = max(stage_times, key=stage_times.get)
    print("\nTotal Execution Time:", round(total, 2), "seconds")
    print("Average Stage Time:", round(average, 2), "seconds")
    print("Fastest Stage:", fastest)
    print("Slowest Stage:", slowest)
def display_results():
    print("\n========== PIPELINE RESULTS ==========")
    total = sum(stage_times.values())
    for name, duration in stage_times.items():
        percentage = (duration / total) * 100
        print("\nStage:", name)
        print("Status:", stage_status[name])
        print("Execution Time:", round(duration, 2), "seconds")
        print("Time Percentage:", round(percentage, 2), "%")
    bottleneck = max(stage_times, key=stage_times.get
    print("\n========== BOTTLENECK ANALYSIS ==========")
    print("Bottleneck Stage:", bottleneck)
    print("Bottleneck Time:", round(stage_times[bottleneck], 2), "seconds")
    print("Recommendation: Optimize the slowest stage.")
    calculate_statistics()
    print("\nPipeline execution completed.")
```

```
def main():
    display_header()
    execute_pipeline()
    display_results()
if __name__ == "__main__":
    main()
```



8. EXPERIMENT RESULTS

```
==========================================
DELIVERY PIPELINE AND IDENTIFYING BOTTLENECKS
==========================================
Pipeline execution started.

Starting Build Stage...
Build completed successfully!
Execution Time: 2.00 seconds

Starting Test Stage...
Test completed successfully!
Execution Time: 4.00 seconds

Starting Package Stage...
Package completed successfully!
Execution Time: 1.00 seconds

Starting Deploy Stage...
Deploy completed successfully!
Execution Time: 3.00 seconds
```

```
========== PIPELINE RESULTS ==========
Stage: Build      - 2.00 seconds (20.0%)
Stage: Test       - 4.00 seconds (40.0%)
Stage: Package    - 1.00 seconds (10.0%)
Stage: Deploy     - 3.00 seconds (30.0%)

Total Execution Time : 10.00 seconds
Bottleneck Stage     : Test
Bottleneck Time      : 4.00 seconds

Recommendation:
Optimize the Test stage.

Pipeline execution completed.
```

Pipeline Stage	Execution Time	Result
Build	2 seconds	Success
Test	4 seconds	Success
Package	1 second	Success
Deploy	3 seconds	Success
Total	**10 seconds**	**4 stages completed**

Total Pipeline Time	:	10 seconds
Average Stage Time	:	2.5 seconds
Fastest Stage	:	Package
Slowest Stage	:	Test
Identified Bottleneck	:	**Test Stage**



9. CONCLUSION

The project **"Delivery Pipeline and Identifying Bottlenecks"** demonstrates the importance of analyzing software delivery performance in a DevOps environment. A delivery pipeline automates the process of moving software changes through different stages, including building, testing, packaging, and deployment. Although automation improves efficiency, delays in individual stages can affect the overall delivery process. The proposed project uses Python to simulate a delivery pipeline and measure the execution time of each stage. The measured durations are stored and compared to identify the stage with the highest execution time.

The experimental example identifies the Testing Stage as the primary potential bottleneck because it has the longest simulated execution time. This approach demonstrates how basic programming concepts such as functions, dictionaries, loops, time measurement, and comparison operations can be applied to DevOps performance analysis. The project also provides an understanding of the importance of CI/CD automation, stage-wise monitoring, and performance optimization.

The basic implementation is suitable for academic purposes and can be extended to real delivery pipelines using Jenkins and Git. Additional monitoring features can help analyze historical performance and identify recurring delays.

However, the longest stage is not always the cause of poor pipeline performance. Actual bottleneck analysis may require examining resource usage, failure rates, stage dependencies, and execution history.

In conclusion, the project achieves its objective of demonstrating how execution-time measurements can be used to identify potential delivery pipeline bottlenecks. It provides a foundation for developing more advanced DevOps performance monitoring and optimization systems.

UGC AUTONOMOUS

10.FUTURE ENHANCMENTS

- **Real-Time Monitoring:** Monitor pipeline performance in real time.
- **Automated Bottleneck Detection:** Identify slow stages automatically.
- **Parallel Execution:** Run multiple tasks simultaneously to reduce execution time.
- **CI/CD Integration:** Integrate with Jenkins and GitHub Actions.
- **Performance Reports:** Generate reports and graphs for pipeline analysis.
- **Cloud Deployment:** Deploy applications using cloud platforms.

ST. MARTIN'S ENGINEERING COLLEGE
SMEC
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

11.REFERENCES

1. **Jenkins Documentation.** *Jenkins User Documentation.* https://www.jenkins.io/doc/
2. **Jenkins Documentation.** *Pipeline.* https://www.jenkins.io/doc/book/pipeline/
3. **Git Documentation.** *Git Reference and User Manual.* https://git-scm.com/doc
4. **Python Software Foundation.** *Python 3 Documentation.* https://docs.python.org/3/
5. **Python Software Foundation.** *Time — Time Access and Conversions.* https://docs.python.org/3/library/time.html
6. **DORA.** *Continuous Delivery Capabilities.* https://dora.dev/capabilities/continuous-delivery/
7. **Atlassian.** *Continuous Integration.* https://www.atlassian.com/continuous-delivery/continuous-integration
8. **GitHub Documentation.** *GitHub Actions.* https://docs.github.com/en/actions
9. **Docker Documentation.** *Docker Documentation.* https://docs.docker.com/
10. **Microsoft Learn.** *Visual Studio Code Documentation.* https://code.visualstudio.com/docs

UGC AUTONOMOUS